

P2636R2

References to ranges should always be viewable

Published Proposal, 2023-01-14

Author:

[Hannes Hauswedell \(deCODE Genetics\)](#)

Audience:

LEWG, SG9

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that the ``views::all`` adaptor and the ``viewable_range`` concept accept lvalue references to all ranges (including move-only views).

Table of Contents

1 Revision History

- 1.1 Revision 2 - 2023-01-14
- 1.2 Revision 1 - 2022-10-14
- 1.3 Revision 0 - 2022-09-14

2 Introduction

- 2.1 The problem (TL;DR)
- 2.2 The solution (TL;DR)

3 Detailed examples

- 3.1 Views from lvalue reference

- 3.2 View composition (starting from lvalue reference)
- 3.3 Views from rvalues
- 3.4 View composition (starting from rvalue)
- 3.5 Generic code

4 Proposal

- 4.1 Wording
- 4.2 Impact

5 "Workarounds"

- 5.1 `std::views::ref`
- 5.2 `std::views::maybe_ref`

6 Lifetime

- 6.1 Returning views on local variables
- 6.2 Returning views on in-parameters
 - 6.2.1 Capturing in-parameter as lvalue
 - 6.2.2 Splitting range composition into multiple lines
 - 6.2.3 Forwarding the in-parameter
 - 6.2.4 Splitting range composition into multiple lines + FWDing

7 Implication for the "meaning" of views

- 7.1 P0896: The One Ranges proposal
- 7.2 P1456 → LWG3481 → P2415 (current state)
- 7.3 Conclusion regarding "meaning"

8 Summary

9 Acknowledgments

10 References

§ 1. Revision History

§ 1.1. Revision 2 - 2023-01-14

- Rewrote and expanded the section [Implication for the "meaning" of views](#).

- Expanded the section on [Lifetime](#) to include many more examples.
- Added [std::views::maybe_ref](#) to the Workarounds section.
- Renamed the types in the examples for easier understanding.
- Fixed various issues that Barry Revzin reported.
- Removed "7.3. P2387" again which was available in a draft of this paper.
- **There was no consensus on pursuing the changes proposed here**, so this revision merely documents the last state of the proposal.

§ 1.2. Revision 1 - 2022-10-14

- Some language corrections and smaller additions to text.
- Added "[The solution \(TL;DR\)](#)".
- Added "[Implication for the "meaning" of views](#)".
- Added section on [std::views::ref](#).
- Feature test macro added to proposed wording.

§ 1.3. Revision 0 - 2022-09-14

- Initial version.

§ 2. Introduction

When dealing with views, two concepts are important: [view](#) ("ranges that are views themselves") and [viewable_range](#) ("ranges that can be turned into views"). Initially, the requirements for the first were very strict, encompassing only non-owning, semi-regular types. The second concept encompassed "views and any lvalue-references to ranges".

Over time, both concepts were relaxed considerably. Views can now be move-only ([P1456](#)) and owning ([P2415](#)), rvalues of non-views are now viewable—in fact most ranges people encounter are now viewable. However, in one particular case, the concepts are now stricter than before: lvalue-references to certain ranges, move-only views, are not viewable. This means that while almost all non-view ranges can be easily composed with view adaptors, certain views now do not qualify for this.

This issue was briefly discussed on the LWG reflector in 2020 where another member highlighted the wide-ranging impact of this change:

I'm concerned about having components that work with ranges but not views, given that views are supposed to be just ranges with additional properties.

A possible "fix" was mentioned in that discussion:

I can envision an alternate design in which `views::all(E)` does admit move-only lvalue views [...] I think that design change needs LEWG approval [...]

But apparently there was no support for this change or no will to pursue it further. This paper picks up that discussion and argues why such a change would indeed make a lot of sense.

§ 2.1. The problem (TL;DR)

```
void foobar(ranges::forward_range auto && r)
{
    auto v = r | views::take(3);
}
```

In the "one ranges proposal" and in multiple shipped compiler versions, the above code was well-formed for all `forward_ranges`. Since P1456, the function template is now unconstrained and will become ill-formed for certain inputs.

§ 2.2. The solution (TL;DR)

Change the definition of `views::all` (and `viewable_range`) so that:

- rvalues are moved
- lvalues are copied if that is possible and cheap
- other lvalues are wrapped in `ref_view` ~~except lvalues to views~~

This means that all lvalue ranges become viewable. The only non-viewable ranges are non-movable rvalues.

§ 3. Detailed examples

To illustrate the different behaviours, I have chosen to provide examples that start out with (1) a copyable view, (2) a copyable non-view range, (3) a move-only non-view range, and (4) a move-only view.

As example types for non-copyable ranges, I use `MoveOnlyRange` and `MoveOnlyView`.

`MoveOnlyRange` could refer to a user-provided type that generates objects upon iteration (e.g. records of data from a stream); it is a move-only single-pass input-range.

The type `MoveOnlyView` is the same as `MoveOnlyRange` except that it also inherits from the empty base class `ranges::view_base`. This has no influence on the semantics of the type other than it begin treated as a view by current standard libraries. While we might suggest to users that such ranges should always be implemented as views, we cannot assume that this will always be the case (there are ranges that are neither containers nor views!).

NOTE, that these examples are also discussed in [the presentation I gave in Kona](#).

§ 3.1. Views from lvalue reference

(1)	<pre>string_view s{"foobar"}; auto v = s views::take(3);</pre>	<code>s</code> is <code>decay_copyd</code>
(2)	<pre>string s{"foobar"}; auto v = s views::take(3);</pre>	<code>s</code> is bound by <code>ref_view</code>
(3)	<pre>MoveOnlyRange g{}; auto v = g views::take(3);</pre>	<code>g</code> is bound by <code>ref_view</code>
(4)	<pre>MoveOnlyView g{}; auto v = g views::take(3);</pre>	currently ill-formed

Before P1456 and P2415, all of these snippets would have been well-formed. Currently, (4) is not, because lvalue references to move-only views are not viewable.

This also means that code is rejected by current compilers that was accepted by previous versions of the same compiler (GCC, MSVC). While this is true for other C++20 features, the particular changes to the view concepts were discussed mainly as relaxing constraints and not as introducing new ones.

§ 3.2. View composition (starting from lvalue reference)

(1)	<pre>string_view s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	<pre>s is decay_copyd v1 is decay_copyd</pre>
(2)	<pre>string s{"foobar"}; auto v1 = s views::take(3); auto v2 = v1 views::transform(**/);</pre>	<pre>s is bound by ref_view v1 is decay_copyd</pre>
(3)	<pre>MoveOnlyRange g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	<pre>g is bound by ref_view v1 is decay_copyd</pre>
(4)	<pre>MoveOnlyView g{}; auto v1 = g views::take(3); auto v2 = v1 views::transform(**/);</pre>	<pre>currently ill-formed</pre>

(4) is still ill-formed; the rest are well-formed.

§ 3.3. Views from rvalues

(1)	<pre>auto v = string_view{"foobar"} views::take(3);</pre>	string_view is decay_copyd
(2)	<pre>auto v = string{"foobar"} views::take(3);</pre>	string is bound by owning_view
(3)	<pre>auto v = MoveOnlyRange{} views::take(3);</pre>	MoveOnlyRange{} is bound by owning_view
(4)	<pre>auto v = MoveOnlyView{} views::take(3);</pre>	MoveOnlyView{} is decay_copyd (actually a move)

All of these snippets are currently well-formed code. In the original ranges proposal, only (1) would have been well-formed, because the others are rvalues of non-views; (4) would not have been regarded as a view, because it is move-only.

§ 3.4. View composition (starting from rvalue)

(1)	<pre>auto v1 = string_view{"foobar"} views::take(3); auto v2 = v1 views::transform(**/);</pre>	string_view is decay_copyd v1 is decay_copyd
(2)	<pre>auto v1 = string{"foobar"} views::take(3); auto v2 = v1 views::transform(**/);</pre>	string is bound by owning_view line 2 is currently ill-formed
(3)	<pre>auto v1 = MoveOnlyRange{} views::take(3); auto v2 = v1 views::transform(**/);</pre>	MoveOnlyRange{} is bound by owning_view line 2 is currently ill-formed
(4)	<pre>auto v1 = MoveOnlyView{} views::take(3);</pre>	MoveOnlyView{} is decay_copyd

```
auto v2 = v1 | views::transform(**/);
```

(actually a move)

line 2 is currently
ill-formed

(2)-(4) are not well-formed, because `v1` in the respective snippet is move-only. Note, how the well-formed snippets in this example are different from [those created from lvalues](#). If the `| views::transform(**/)` had been appended to the first line and no `v2` created, it would have worked. This means that—contrary to what many previous ranges guides suggest—it makes an important difference now whether you compose the view incrementally or in one line. And it appears that "creating in one line" needs to be recommended.

§ 3.5. Generic code

Returning to the problem of generic code, assume the following:

(definitions for `to_upper` and `to_lower` omitted)

```
void print_up_low(ranges::forward_range auto && rng)
{
    for (char c : rng | views::transform(to_upper))
        cout << c;
    cout << '\n';
}
```

```

    for (char c : rng | views::transform(to_lower))
        cout << c;
}

```

(1a)	<pre> print_up_low(string_view{"foobar"}); print_up_low(string_view{"foobar"} views::take(3)); </pre>	both calls are well- formed
(2a)	<pre> print_up_low(string{"foobar"}); print_up_low(string{"foobar"} views::take(3)); </pre>	first call is well-formed second call is ill-formed
(1b)	<pre> string_view s{"foobar"}; print_up_low(s); print_up_low(s views::take(3)); </pre>	both calls are well- formed
(2b)	<pre> string s{"foobar"}; print_up_low(s); print_up_low(s views::take(3)); </pre>	both calls are well- formed

These examples show that it is easy to run into problems with "move-only views", even when just using simple adaptors on types likes strings and string_views. Is the behaviour shown above intended? Do we expect developers to understand why 1 of 8 calls to `print_up_low()` is ill-formed and why?

Curiously, it is now the example of "creating a view in one line" that is ill-formed, while previous examples suggested that exactly this usage pattern increases the likelihood of the code being well-formed.

§ 4. Proposal

This papers suggests **making all previously shown ill-formed snippets well-formed** by changing the `viewable_range` concept and the definition of the `views::all` adaptor.

§ 4.1. Wording

[26.4.5 Other range refinements \[range.refinements\]](#)

Change the definition of `viewable_range` to:

```
template<class T>
concept viewable_range =
    range<T> &&
    ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>
    (!view<remove_cvref_t<T>> && (is_lvalue_reference_v<T> || (movable<remove
    is_lvalue_reference_v<T> || (movable<remove_reference_t<T>> && !is-initia
```

[\[26.7.6.1 General \[range.all.general\]\]](#)

Change the definition of `views:all` to

- (2.1) decay-copy(E) if that expression is well-formed and the decayed type of E models view.
- (2.2) Otherwise, `ref_view{E}` if that expression is well-formed.
- (2.3) Otherwise, `owning_view{E}`.

[\[17.3.2 version.syn\]](#)

Bump the `__cpp_lib_ranges` version.

§ 4.2. Impact

These changes are *breaking* insofar as all concept changes are always potentially breaking. However, this proposal very clearly relaxes the constraints on `viewable_range` so the scope is broadened, and in practice more code will work than before and not less. In fact, this will *unbreak* code that was well-formed in earlier published versions of the IS (and is well-formed in released versions of GCC and MSVC) but that has been broken in the meantime.

Should LEWG accept this proposal for C++23, it could ask LWG to denote it as a C++20 defect fix. There is precedent for this, because the `view` and `viewable_range` concepts have already been

changed multiple times with wide-ranging modifications being back-ported to C++20 and even older compiler versions (GCC10, GCC11); although technically these changes were feature changes more than fixes. The current changes are much smaller in scope and re-establish initial ranges behaviour.

§ 5. "Workarounds"

For the previously shown ill-formed snippets, it is possible to work around the problem by any or either of the following methods:

1. `move`-ing the initial range into the pipe (resulting in an `decay_copy` or `owning_view`); or
2. manually wrapping the initial range in `ref_view`

Since most users never directly interact with `views::all`, `ref_view` and `owning_view`, I assume that it will be very hard to explain where (and which of) these workarounds are necessary.

Furthermore, this creates a burden on generic code that will either need to `if constexpr` on ranges passed to it by reference or blindly wrap them in `ref_view` to make them viewable.

Note also that "just adding `ranges::ref_view`" in a generic context may have undesired side-effects. Even for ranges that are already viewable, it is not a no-op; e.g. it prevents copyable views from being captured by value which changes their behaviour with regard to `const` (they are now shallow const but would have been deep const before). It also changes type-specific behaviour in view adaptor objects like the "same-type optimisations" in `views::take` and `views::drop`.

§ 5.1. `std::views::ref`

It was suggested that having `views::ref` as a shortcut for `ranges::ref_view` would alleviate the problems presented here. While I do not object to having such a view factory, I do not see how it solves the fundamental problem of a bad default.

It is of no consequence to almost all algorithms whether they are operating on a move-only view or a copyable range, so we shouldn't require a workaround.

§ 5.2. `std::views::maybe_ref`

During the Kona meeting, Barry Revzin proposed a range adaptor closure object that acts like `views::all` for all viewable ranges and like (a potential) `views::ref` for non-viewable ranges, in particular move-only views.

This avoids the indirection for copyable views, but it does not solve the fundamental problem of a bad default. Such an adaptor would need to be inserted into every generic algorithm that uses range composition.

If that is going to be the official recommendation, why can't we just make `views::all` behave in that way by default?

§ 6. Lifetime

Previous mailing list comments suggested that the changes proposed here would create issues with view lifetime. The following three examples explore some of those implications.

§ 6.1. Returning views on local variables

Nr.	Code	Status quo	This proposal
(1)	<pre>string_view s{"foobar"}; return s views::take(3);</pre>	not dangling	not dangling
(2)	<pre>string s{"foobar"}; return s views::take(3);</pre>	dangling	dangling
(3)	<pre>MoveOnlyRange g{}; return g views::take(3);</pre>	dangling	dangling
(4)	<pre>MoveOnlyView g{}; return g views::take(3);</pre>	ill-formed	dangling

Returning a view created from a local variable can create a dangling reference. This has always been the case and is inherent to `ranges::ref_view` (and independent of the underlying range). While it could be argued that preventing it for *some ranges* is beneficial, it is unclear why this should only be the case for move-only views and not other ranges; especially since example (2) is likely to appear more often than example (4) and the range in (3) is semantically identical to the range in (4).

On a sidenote: Now that we have move-only owning views, one could make an argument for making all uses of `ranges::ref_view` explicit (removing the option from `views::all` and recommending

that users `move` into a view composition by default or explicitly opt into having a reference if desired). However, such a change would be quite disruptive, so pursuing this argument at this point seems futile.

§ 6.2. Returning views on in-parameters

§ 6.2.1. Capturing in-parameter as lvalue

```
auto foo(auto && rng)
{
    return rng | views::take(3) | views::transform(**/);
}
```

Nr.	<code>decltype(rng)</code>	Status quo	This proposal
(1a)	<code>string_view &</code>	not dangling	not dangling
(1b)	<code>string_view &&</code>	not dangling	not dangling
(2a)	<code>string &</code>	not dangling	not dangling
(2b)	<code>string &&</code>	dangling	dangling
(3a)	<code>MoveOnlyRange &</code>	not dangling	not dangling
(3b)	<code>MoveOnlyRange &&</code>	dangling	dangling
(4a)	<code>MoveOnlyView &</code>	ill-formed	well-formed & not dangling
(4b)	<code>MoveOnlyView &&</code>	ill-formed	dangling

The rvalue cases are essentially the same as in the previous example, but for the lvalue cases one would become well-formed (and not dangling) with the changes proposed here.

§ 6.2.2. Splitting range composition into multiple lines

```
auto foo(auto && rng)
{
    auto v = rng | views::take(3);
```

```
return v | views::transform(**/);
}
```

Nr.	<code>decltype(rng)</code>	Status quo	This proposal
(1a)	<code>string_view &</code>	not dangling	not dangling
(1b)	<code>string_view &&</code>	not dangling	not dangling
(2a)	<code>string &</code>	not dangling	not dangling
(2b)	<code>string &&</code>	dangling	dangling
(3a)	<code>MoveOnlyRange &</code>	not dangling	not dangling
(3b)	<code>MoveOnlyRange &&</code>	dangling	dangling
(4a)	<code>MoveOnlyView &</code>	ill-formed	well-formed & not dangling
(4b)	<code>MoveOnlyView &&</code>	ill-formed	dangling

The concern was raised that splitting a single statement into multiple ones would create dangling cases from previously ill-formed cases. When capturing the range as lvalue from an in-parameter (which may be an lvalue or rvalue itself), this is not the case; the same cases are ill-formed/dangling as before.

§ 6.2.3. Forwarding the in-parameter

```
auto foo(auto && rng)
{
    return FWD(rng) | views::take(3) | views::transform(**/);
}
```

Nr.	<code>decltype(rng)</code>	Status quo	This proposal
(1a)	<code>string_view &</code>	not dangling	not dangling
(1b)	<code>string_view &&</code>	not dangling	not dangling
(2a)	<code>string &</code>	not dangling	not dangling
(2b)	<code>string &&</code>	not dangling	not dangling
(3a)	<code>MoveOnlyRange &</code>	not dangling	not dangling
(3b)	<code>MoveOnlyRange &&</code>	not dangling	not dangling
(4a)	<code>MoveOnlyView &</code>	not dangling	not dangling
(4b)	<code>MoveOnlyView &&</code>	not dangling	not dangling

Although this is not the usually recommended form of invoking range compositions, forwarding into the range composition fixes all the dangling cases. This proposal does not affect that.

§ 6.2.4. Splitting range composition into multiple lines + FWDing

```
auto foo(auto && rng)
{
    auto v = FWD(rng) | views::take(3);
    return v | views::transform(/**/);
}
```

Nr.	<code>decltype(rng)</code>	Status quo	This proposal
(1a)	<code>string_view &</code>	not dangling	not dangling
(1b)	<code>string_view &&</code>	not dangling	not dangling
(2a)	<code>string &</code>	not dangling	not dangling
(2b)	<code>string &&</code>	ill-formed	dangling
(3a)	<code>MoveOnlyRange &</code>	not dangling	not dangling
(3b)	<code>MoveOnlyRange &&</code>	ill-formed	dangling
(4a)	<code>MoveOnlyView &</code>	ill-formed	well-formed & not dangling
(4b)	<code>MoveOnlyView &&</code>	ill-formed	dangling

The concern was raised that splitting a single statement into multiple ones would create dangling cases from previously ill-formed cases. When we forward or move values into the range composition, this is true. However, we need to be aware of three things here:

1. The recommended way of dealing with ranges in generic code, is to capture by `&&` and pass on as lvalue, not to forward/move (see also [below](#)). Since that is the easier/shorter syntax, it is reasonable to assume that developers who make a choice to forward/move, do so intentionally and know what they are doing.
2. If you explicitly forward/move something into the initialising expression of a local variable, it is obvious that that variable can take ownership of the respective dynamic memory; that's the whole point.

3. There is a very easy fix: `move(v)` into the returning expression, as well; this will make all examples well-formed and non-dangling. It is just a matter of forwarding/moving consistently. Either you don't do it all, or you do it for all "lines" of the range composition.

§ 7. Implication for the "meaning" of views

Objections to this proposal include that it would distort the meaning of views and that »[t]he entire point of a view is that this is something that should be taken by value and owned«.

NOTE, that this proposal does not change the definition of `view` at all!

So what is implied here, is that the meaning of `view` is actually hidden in the definition of `viewable_range` (which itself uses the definition of `view`...). We will encounter this logical problem again below.

In any case, I disagree with the objection raised and want to argue instead that (a) views are not *generally* taken by value; and (b) the view concept is not useful (in most generic code) anyway.

In fact, part of this argument is already made in [P2415](#):

The initial thought [...] might be that [...] we write algorithms that take views by value: [...] and we definitely do encourage people to take *specific* views by value - such as `span` and `string_view` [...]. Either way, we very much want range-based algorithms to be able to operate on, well, ranges, so these are always written instead to take ranges by forwarding reference:

```
template <input_range R>
void some_algo(R&& r);
```

→ we do not actually take views by value in generic code, and we do not use the view concept to constrain algorithms.



As far as "the meaning of views" is concerned, I would like to have a closer look how this changed from early proposals to the current state, and how "views" and "range composition" are related.

Evidently, `viewable_range` is used to constrain "what goes into" a range composition, and `view` "what comes out", so the implementors of range composition facilities (need to) care about these

concepts. But it does not yet provide evidence of their meaning to the users of range composition.

It should also be noted that there are views that are independent of range composition (e.g. `span`, `string_view`), and that range factories (e.g. `views::iota`) can be used standalone, as well.

§ 7.1. P0896: The One Ranges proposal

P0896: The One Ranges proposal

1. `ranges::view`: a semi-regular range with $O(1)$ operations
2. `ranges::viewable_range`: any lvalue of a range (including views) or rvalue of a view

The definition of view is very straight-forward here; in particular the notion of "copying something in constant time" is easy to explain. Views were often summarised as "light-weight ranges" or "non-owning ranges". Furthermore, it is easy to illustrate these properties by using `string_view` and `span` (the simplest views) as examples. Importantly, one does not need to refer to "range composition" at all, in explaining what a view is.

Since it was/is recommended, in algorithms code, to pass ranges by lvalue into range compositions, the role of `ranges::viewable_range` is purely technical. Users neither constrain their algorithms with it, nor do they explicitly try to make their types model this concept. The meaning of `viewable_range` is not obvious, but it does not need to be; as far as users are concerned, it could have been exposition-only.

Generic ranges algorithms were not (and are not) constrained with the `view` concept, but its "cheap copy" properties might allow its use e.g. as constraints in contexts that rely on pass-by-value. Furthermore, views were the only ranges that you could pass to range compositions as rvalues, so there was a clear incentive/benefit to opting into the `view` concept for a user-defined type: it enabled usage patterns that were not possible for types that did not model `view`.

§ 7.2. P1456 → LWG3481 → P2415 (current state)

P1456: Move-only views

LWG3481: viewable_range mishandles lvalue move-only views

P2415: What is a view?

1. `ranges::view`: a range that is copyable in $O(1)$ or not copyable at all (more technical requirements omitted)

2. `ranges::viewable_range`: any rvalue of a movable range (including views) or lvalue of range except move-only views

Views, the types of objects returned by range compositions, can now own their elements and need no longer be copyable at all. This makes it more difficult to explain what a view is, and P2415 can only give this answer:

There may not be a clean answer, which is admittedly unsatisfying, but it mainly boils down to:

```
auto rng = v | views::reverse;
```

If `v` is an lvalue, do you want `rng` to *copy* `v` or to *refer* to `v`? If you want it to copy `v`, because copying `v` is cheap and you want to avoid paying for indirection and potential dangling, then `v` is a **view**. If you want to refer to `v`, because copying `v` is expensive [...], then `v` is not a **view**.

This has two important implications:

1. It is no longer possible to define **view** without referring to range composition. The definition thus becomes somewhat circular; "a view is something that gets special treatment by a view".
2. The defining distinction made (capturing lvalue view by copy instead of ref) is *very* technical. In practice, it would be barely noticeable for the average user if the distinction were not made; i.e. you could just capture the view by ref, too.¹ Especially in generic code, the user already opts into the cost for "indirection and potential dangling" by accepting lvalue non-views.²

¹ I am not arguing that this should be done, but I am saying that distinguishing view/non-view solely based on this... is a bit underwhelming.

² If the user wishes to avoid all chance of indirection and dangling, they have to move `v` into the range composition.

All empirical evidence that I have from showing examples to C++ experts and discussing this matter, is that the average C++ programmer will never understand what the **view** concept entails. This sounds dire, but maybe it doesn't have to be? Maybe users can just use range composition and the views-machinery without understanding the view concept? I think the chances for this are quite good. Since almost all ranges can now be "put into" range compositions (also rvalues of non-views), there is no incentive to make user-defined types model the **view** concept. It will "just work" without, and people will not use the **view** concept in their own code.

However, there is one showstopper: The `viewable_range` concept is no longer "just a technicality". As shown previously, users do need to be aware of and understand the `viewable_range` concept,

AND use it to constrain their algorithms (or pick other obscure workarounds that are only comprehensible if one understands the concepts).

§ 7.3. Conclusion regarding "meaning"

It is possible to envision range composition that allows composing all ranges that are composable right now (in particular move-only ranges and rvalues of containers), but that does not bend the view concept into its current form. I am exploring such a design in standalone library, but even if I were able to convince LEWG of this design, it is too late to change the standard library range adaptors in such a fundamental way now.

The only way forward, is to acknowledge that, both, the `view` concept and `viewable_range` concept are implementation detail of the standard's "range composition" machinery. Consequently, our mission should not be to make users understand their meaning, but to make sure they don't need to.

To achieve this, we need to apply the changes proposed in this paper and make sure that the ergonomic and ubiquitous syntax of `r | views::take(3)` works for all lvalue ranges (again).

§ 8. Summary

Treating move-only views as fundamentally different from copyable views when evaluating viewability, creates a source of user error and frustration. I agree with the changes made to views, but we now have problems explaining what a view even is ([P2415](#)); having two sets of views with different usage patterns definitely won't help. This does not only affect exotic, user-defined types, but can become obvious even in toy examples created with strings.

The difference becomes even more pronounced with standalone move-only ranges that can be implemented as views but where we cannot assume that they always will be. For those types, it is unclear what benefit **opting into** the view concept provides—if that only makes the type **opt out** of easy view composition.

Further complications arise in generic code, which doesn't care about the view concept or ownership at all and could previously assume that all lvalue ranges are viewable. It now needs workarounds which are easy to forget and come with their own side-effects.

While it is true that all direct and indirect uses of `ranges::ref_view` entail the possibility of dangling references, it is unclear why this is considered more dangerous for move-only views than other ranges. I hope to have alleviated some concerns in this regard and shown that the proposed increase in consistency and usability provides a clear benefit.

§ 9. Acknowledgments

I want to thank Barry Revzin and Tim Song for their work on revising (and adding lots of!) views and for their comments on the current topic. Thanks also to several members of the German WG21 / DIN group for feedback on this paper.

Thanks to my employer, deCODE Genetics, for sponsoring my WG21 time.

§ 10. References

Referenced papers:

- [P0896: The One Ranges proposal](#)
- [P1456: Move-only views](#)
- [LWG3481: viewable_range mishandles lvalue move-only views](#)
- [P2415: What is a view?](#)
- [P2387: Pipe support for user-defined range adaptors](#)

On the topic of this paper:

- [Presentation I gave in Kona](#)
- Short 2020 discussion in LWG: <https://lists.isocpp.org/lib/2020/09/17353.php>
- Short 2022 discussion in LWG: <https://lists.isocpp.org/lib/2022/08/23438.php>
- Brief mention in minutes: <https://wiki.edg.com/bin/view/Wg21telecons2021/LWG-issues-20210115>
- GCC issue with other examples: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=106669